

Question 1

Part One: Regular Expressions

L is the binary strings that do not contain the substring 11.

$A = \{0,1\}$

- Strings that are in language L: 010, 101, 010101, 101010, ... etc.
- String that are not in language L: 110, 011, 1111, 00111, .. etc
- Regular expression for L is $(0 | 10)^* (\epsilon | 1)$,
Here $(0|10)$ will not allow to have substring 11.
- Regular expression for all the binary strings that contain at least the substring 11.
 $(0|1)^* 11 (0|1)^*$
The middle 11 will always be in the string.

Part Two: Context Free grammar

- Derivation of the string $\{\}\{\}\{\}$

Question 1: Part 2: Context Free Grammar

Solⁿ:

Production rule, $S \rightarrow SS \mid \{S\} \mid [S] \mid \epsilon$

Where S is start symbol.

5) $S \rightarrow \{S\}$ (∵ Apply $S \rightarrow \{S\}$)

$S \rightarrow \{SS\}$ (∵ Apply $S \rightarrow SS$)

$S \rightarrow \{[S]S\}$ (∵ Apply $S \rightarrow [S]$)

$S \rightarrow \{[S][S]\}$ (∵ Apply $S \rightarrow [S]$)

$S \rightarrow \{[\epsilon][S]\}$ (∵ Apply $S \rightarrow \epsilon$)

$S \rightarrow \{[\epsilon][\epsilon]\}$ (∵ Apply $S \rightarrow \epsilon$)

$S \rightarrow \{[][\epsilon]\}$

- The given grammar generate the set of all strings with balanced braces and square brackets (arbitrary nesting). Both { and [must have matching closing symbols } and], strings like $\{\}$, $\{\{\}\}$, $\{\}\{\}$ are valid as well as empty string is also valid.

- c. Modifying the grammar such a way that prevent crossing brackets like $\{\}$
- $$S \rightarrow A \mid B \mid AB \mid BA \mid \varepsilon$$
- $$A \rightarrow \{A\} \quad (\text{braces-only section})$$
- $$B \rightarrow [B] \quad (\text{brackets-only section})$$
- It separates brace sections A from bracket sections (B). Valid strings like $\{\}\{\}$, $\{\}\{\}\{\}$, etc.

Part Three

- a. Parse tree and Syntax tree for $(a+b)*c$

Question 1 : Part 3 : Parse vs Syntax tree

So Iⁿ:

Grammar $E \rightarrow E + E \mid E * E \mid (E) \mid a \mid b \mid c$

where E is start symbol

So, for $(a+b)*c$

$$E \rightarrow E * E \quad (\because E \rightarrow E * E)$$

$$E \rightarrow (E) * E \quad (\because E \rightarrow (E))$$

$$E \rightarrow (E + E) * E \quad (\because E \rightarrow E + E)$$

$$E \rightarrow (a + E) * E \quad (\because E \rightarrow a)$$

$$E \rightarrow (a + b) * E \quad (\because E \rightarrow b)$$

$$E \rightarrow (a + b) * c \quad (\because E \rightarrow c)$$

(8)

Parse Tree

Syntax Tree

- b. Role of internal nodes and the leaf nodes

Internal nodes: it represent non-terminal symbols in Grammar. They correspond to the intermediate steps in the derivation process. For eg: E, +, *

Leaf Nodes: it represent the terminal symbols in grammar. These are the basic units that make up the expression. For eg: a,b,c

c. Yes this grammar can generate the string $a+b*c$.

Option:1

$E \rightarrow E+E$ apply $E \rightarrow E+E$

$E \rightarrow E + E^*E$ apply $E \rightarrow E^*E$

$E \rightarrow a+E^*E$

$E \rightarrow a+b^*E$

$E \rightarrow a+b*c$

Option 2:

$E \rightarrow E^*E$ apply $E \rightarrow E^*E$

$E \rightarrow E + E^*E$ apply $E \rightarrow E+E$

$E \rightarrow a+E^*E$

$E \rightarrow a+b^*E$

$E \rightarrow a+b*c$

It will be ambiguous because there is more than one way to parse the tree.

Bonus Question

Difference between parse tree and syntax tree in term of GED Algorithm.

➔ The tree can be transformed into the syntax tree using Graph Edit Distance Algorithms by applying the minimum set of edits:

- Delete non-essential nodes: remove all non-terminal E nodes that appear only because of grammar rules.
- Remove extra edges: delete edges connected to those removed E nodes.
- Collapse chains: merge operator nodes directly with their operands.
- Keep only essential nodes: retain only the operator hierarchy and operands

The syntax tree is a pruned, condensed version of the parse tree, preserving only the semantic structure of the expression.

Question 2

Part One: Graph Centrality

- a. Highest Degree Centrality

Person C has highest betweenness centrality, which means C acts as a meeting point for different social groups.

- b. Highest betweenness centrality

C has highest betweenness centrality. Example paths through C:

A → B → C → D

A → B → C → I

A → B → C → F

E → C → D

B → C → M

C plays a central role in maintaining the overall structure of the network. Without C, the network would lose a lot of its cohesion, and the flow of information would be severely impaired.

- c. Lowest betweenness centrality

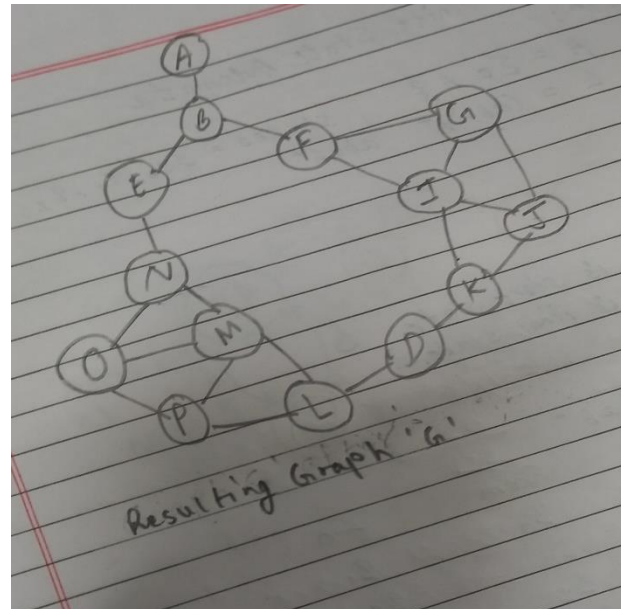
A has lowest betweenness Centrality because only B is connected with it. Removing it has zero impact on network connectivity.

Part Two: Graph Transformation and Graph Edit Distance

- a. By applying to the rule to the graph, C could be represented by P2. The transformation rule defines p2 as a node with a degree of 6. From graph c is the only person with 6 connections.
- b. LHS and RHS of the transformation rule
- LHS: a sequence of three people, p1, p2, p3, where p2 has a degree of 6.
- RHS: delete p2(node with degree 6) and all of its edges

c. Resulting Graph after applying the rule

After applying the rule to C we remove C and all its edges. So, C will no longer have any connections in the graph, and the nodes that were connected to C (B,D,E,F,I,M).



d. The minimum social distance is the smallest number of changes needed to split the graph into multiple communities. This can be analyzed using the Graph Edit Distance algorithm, which considers the cost of deleting edges or nodes disconnect parts of the graph.

After removing C, the remaining graph is already split into multiple components, so the minimum social distance might be 0. This is because C removal already breaks the network into separate communities, meaning no further edits are required to create at least two separate communities.

The minimum social distance is 0 because the removal of C already disconnects the graph into multiple communities.

e. With the new cost structure:

Cost to delete a person = 2

Cost to insert a person = 2

Cost to substitute a person = 2

Cost to delete a friendship = 3

Cost to insert a friendship = 3

No, the minimum social distance will likely to increase because the cost to delete friendships (3) may make it more expensive to fully separate the communities, compared to just deleting a person.

Bonus Question:

Graph Edit Distance Algorithm is implemented in python code and it can not handle big graphs.

Question 3

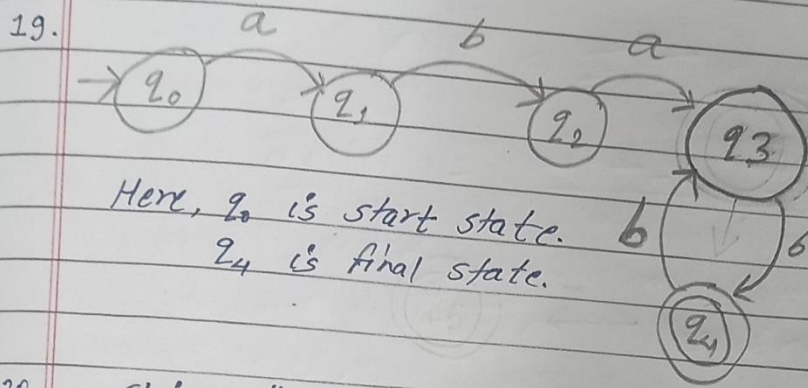
Part One: Finite State Automata

Question 3

Part one: Finite State Automata

Soln:

$A = \{a, b\}$, states = $\{q_0, q_1, q_2, q_3, q_4\}$
 $L = (ab)^2 b^+ abab^+$



Here, q_0 is start state.
 q_4 is final state.

20. String "abab"

input	From	to
a	q_0	q_1
b	q_1	q_2
a	q_2	q_3
b	q_3	q_4

because it ends at q_4 , abab is accepted.

21. string "aba"

input	From	to
a	q_0	q_1
b	q_1	q_2
a	q_2	q_3

q_3 is not final state, so, "aba" is not accepted.

Part Two: Petri Nets

a. Two-Vehicle Petri Net Mechanism for Concurrent Access

We need to model a system where two vehicles can access the intersection in a fair and non-blocking way. To do this, we can use Petri net places and transitions for both vehicles and a shared place to represent the intersection.

Basic Structure:

Each vehicle (Vehicle 1 and Vehicle 2) has the following stages:

- Wait to enter: The vehicle is waiting to access the intersection.
- Occupy: The vehicle occupies the intersection.
- Finish from: The vehicle finishes using the intersection.
- Leave: The vehicle leaves the intersection.

The shared place will be the Intersection place. Only one vehicle can occupy the intersection at a time, so we need to link the vehicles' movements to this shared place.

Petri Net Design:

1. Places:

- Wait1, Occupy1, Finish1, Leave1 (for Vehicle 1).
- Wait2, Occupy2, Finish2, Leave2 (for Vehicle 2).
- Intersection: The shared place where a vehicle can occupy the intersection.

2. Transitions:

- T1: Vehicle 1 moves from Wait1 to Occupy1 if Intersection is free.
- T2: Vehicle 1 moves from Occupy1 to Finish1.
- T3: Vehicle 1 moves from Finish1 to Leave1, and the Intersection becomes free.
- T4: Vehicle 2 moves from Wait2 to Occupy2 if Intersection is free.
- T5: Vehicle 2 moves from Occupy2 to Finish2.
- T6: Vehicle 2 moves from Finish2 to Leave2, freeing the Intersection.

This structure ensures that only one vehicle can occupy the intersection at a time by controlling access through the Intersection place.

b. Conditions for One Vehicle at a Time

To ensure that only one vehicle occupies the intersection at a time, we can impose the following conditions on the Petri net:

- **Intersection Token:** There should only be one token in the Intersection place. If one vehicle is occupying the intersection, a token will be placed in the Intersection place. This prevents the second vehicle from moving into the intersection until the token is removed.
- **Vehicle Access Control:** A vehicle can only move from its Wait place to the Occupy place if the Intersection place is free (i.e., no token is in Intersection).
- **Transition Blockage:** If the Intersection is occupied, the vehicle's transition to Occupy is blocked until the token is removed.

This ensures that only one vehicle can access the intersection at a time, preventing both from occupying the intersection simultaneously.

c. Possibility of Deadlock

In this Petri net mechanism, deadlock is possible under certain conditions if the system is not designed correctly. However, with the proper synchronization of transitions and ensuring that vehicles alternate fairly in accessing the intersection, deadlock can be avoided.

Deadlock could occur if:

- Both vehicles are stuck in the Wait places without making progress because the intersection is always occupied by the other vehicle.

To prevent deadlock, we can introduce a fairness condition where if both vehicles are waiting, one of them is chosen to proceed based on an alternating mechanism or by priority rules. This ensures that at least one vehicle will always move forward, preventing the system from stalling.

Question 4: Ontology

Class Hierarchy

1. **Location** – Represents geographical entities. It has two subclasses:
 - a. **City**, which includes individual cities such as Edinburgh.
 - b. **Country**, which represents nations, e.g., Scotland.
2. **Landmark** – Represents important places or monuments in a city. Subclasses include:
 - a. **Castle** (e.g., Edinburgh Castle)
 - b. **Palace** (e.g., Holyrood Palace)
 - c. **Mountain** (e.g., Arthur's Seat)
3. **Event** – Represents activities or festivals that take place in the city. Subclasses include:
 - a. **Festival** (e.g., Edinburgh Festival Fringe)
 - b. **CulturalEvent** (e.g., Hogmanay, Royal Edinburgh Military Tattoo)
4. **Institution** – Represents organizations located in Edinburgh. Subclasses include:
 - a. **University** (e.g., University of Edinburgh)
 - b. **Museum** (e.g., National Museum of Scotland)
 - c. **Library** (e.g., Edinburgh Public Library)
5. **Neighbourhood** – Represents areas within the city (e.g., Old Town, New Town, Leith).
6. **Person** – Represents notable people associated with Edinburgh. Subclasses include:
 - a. **Scientist** (e.g., Charles Darwin, James Clerk Maxwell)
 - b. **Author** (e.g., J.K. Rowling)

Object and data Properties along with domains and ranges

- 1) **hasLandmark** – Links a city to its landmarks.
 - a. Domain: City
 - b. Range: Landmark
- 2) **hostsEvent** – Links a city to the events that take place in it.
 - a. Domain: City
 - b. Range: Event
- 3) **hasInstitution** – Links a city to institutions located in it.
 - a. Domain: City
 - b. Range: Institution
- 4) **hasNeighbourhood** – Links a city to its neighborhoods.
 - a. Domain: City
 - b. Range: Neighbourhood

- 5) **hasNotablePerson** – Links a city to people associated with it.
 - a. Domain: City
 - b. Range: Person

Data properties store literal values such as names or descriptions:

1. **hasName** – Stores the name of an individual.
 - a. Domain: Thing
 - b. Range: String
2. **hasDescription** – Stores a brief description.
 - a. Domain: Thing
 - b. Range: String

Asserted individuals with usage

1. **City:** Edinburgh is an individual of class City, and it is linked to its landmarks, events, institutions, neighborhoods, and notable people.
2. **Landmarks:** EdinburghCastle (Castle), HolyroodPalace (Palace), and ArthursSeat (Mountain) are individuals of Landmark subclasses and are linked to Edinburgh via hasLandmark.
3. **Events:** EdinburghFestivalFringe (Festival), Hogmanay (CulturalEvent), and RoyalEdinburghMilitaryTattoo (CulturalEvent) are linked to Edinburgh through hostsEvent.
4. **Institutions:** UniversityOfEdinburgh (University), NationalMuseumOfScotland (Museum), and EdinburghPublicLibrary (Library) are linked using hasInstitution.
5. **Neighbourhoods:** OldTown, NewTown, and Leith are individuals linked to Edinburgh via hasNeighbourhood.
6. **Notable people:** CharlesDarwin, JamesClerkMaxwell, and JKRowling are individuals of Person subclasses linked with hasNotablePerson.

Question 5

- a. "all students in class" means $\forall x(Student(x) \wedge InClass(x))$
"are studying" means $Studying(x)$
So,
 $x(InClass(x) \wedge Student(x) \rightarrow Studying(x))$
Meaning, every student who is in the class is studying.
- b. "some teachers" means $\exists x(Teacher(x))$
"teach mathematics" means $Teacher(x, Mathematics)$
So,
 $\exists x(Teacher(x) \wedge Teacher(x, Mathematics))$
It means there exists at least one teacher who teaches mathematics.
- c. $\forall x(Student(x) \wedge InClass(x) \wedge x \neq John \rightarrow TallerThan(John, x))$
John is taller than every other student in the class except himself.
- d. "Every person who owns a car" means $\forall x(Person(x) \wedge OwnsCar(x))$
"Also has a driving license" means $HasDrivingLicense(x)$
So,
 $\forall x(Person(x) \wedge OwnsCar(x) \rightarrow HasDrivingLicence(x))$
It means that If a person owns a car, then that person has a driving license.
- e. "There is a book" means $\exists b(Book(b))$.
"Every student has read" means $\forall x(Student(x) \rightarrow HasRead(x, b))$.
So,
 $\exists b(Book(b) \wedge \forall x(Student(x) \rightarrow HasRead(x, b)))$
This means that there exists a book that all students have read.